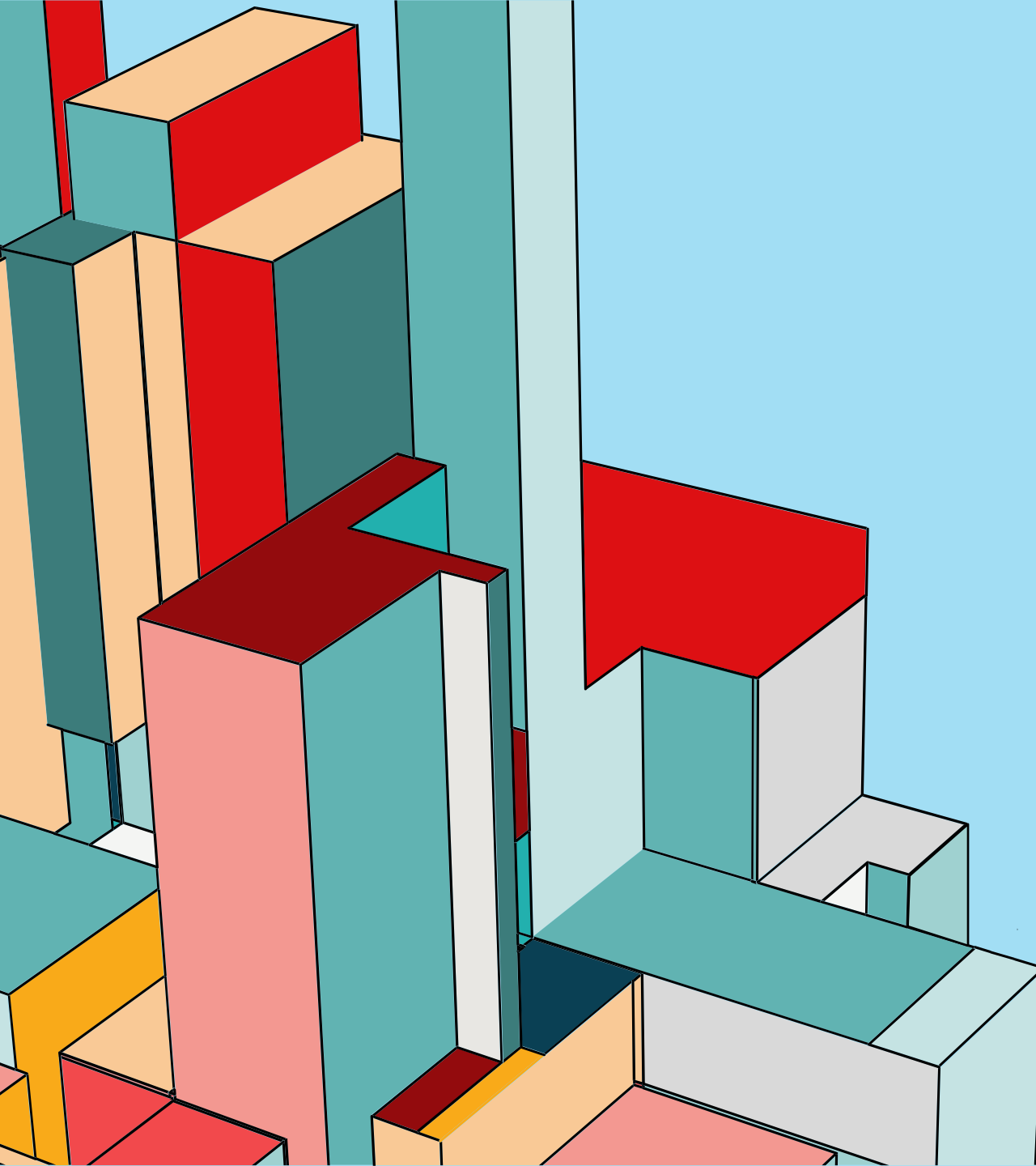


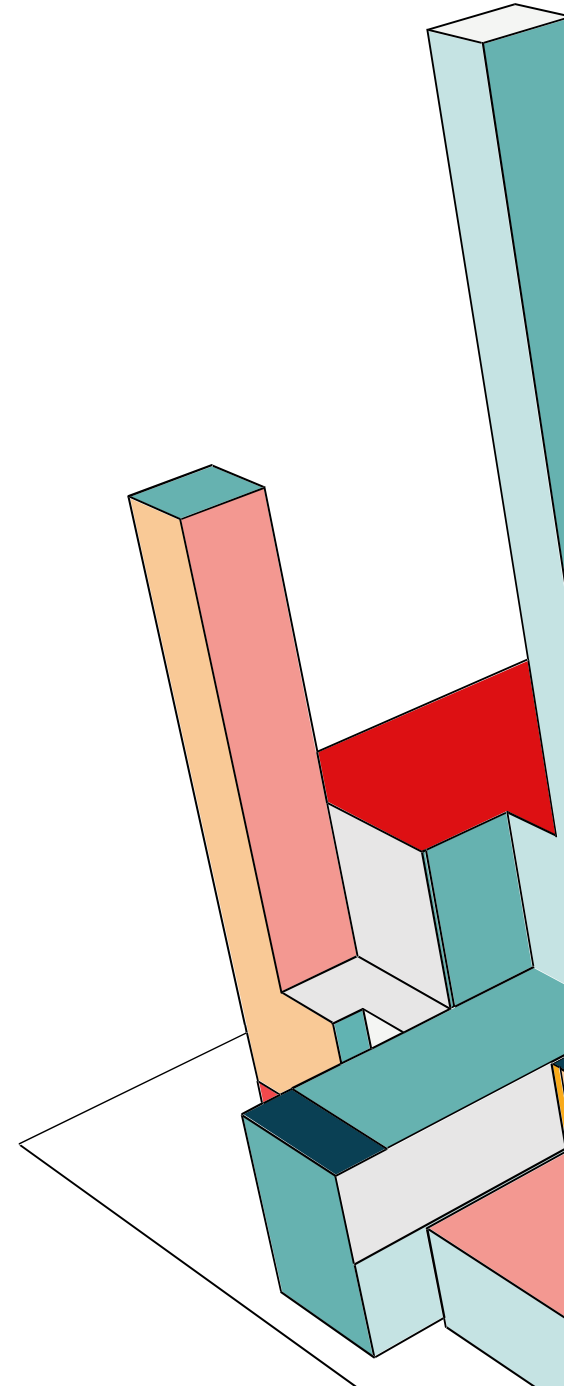


LECTURE 15 **HASHING III & EXAM REVIEW**

CSIS 2300 Introduction to
Algorithms, Spring 2026
Dr. Dan DiTursi

ANNOUNCEMENTS & REMINDERS

- Exam 2 Wednesday 8am (same rooms)
 - Accommodations: DCC 232
 - Last names A-G: DCC 324
 - Last names H-L: DCC 337
 - Last names M-Z: DCC 318
- No lab this week for exam, NOR NEXT WEEK (because of GM Week).



DIGITAL SIGNATURES

These days, sensitive business is conducted online as a matter of routine. If you think about it, this is fairly remarkable: Banks and other financial institutions are generally some of the most cautious businesses out there.

- How does everyone know that the party on the other end is really who they say they are? How can they demonstrate this to, say, a court of law?
- How do they know that the document they are signing really is the one sent by the other party? How can they be sure that it won't be edited later?

3 The surprising answer to these questions is hashing!

DIGITAL SIGNATURES

The main idea goes like this: We take an arbitrary document and apply a publicly known hash function to it, producing a fixed-length bitstring (say, 512 bits) – this is usually called the message *digest*.

The publisher of the document also posts the digest to its webpage. Anyone who obtains a copy of the document elsewhere can run the hash function and verify that they get the same digest.

SUBMITTY POLL #1

If the hash function in the above example is uniform, what is the probability of two randomly selected messages producing the same message digest?



CRYPTOGRAPHIC HASH FUNCTIONS

The purpose of using message digests in this way is to ensure that the document has not been altered *en route*. For example, if we're downloading software, we want to be assured that malware has not been introduced.

Given this goal, what properties must the hash function have?

- Signatures need to be hard to forge!
- In this case, it means that it should be difficult to find / create *another* document that produces the same digest.

CRYPTOGRAPHIC HASH FUNCTIONS

An n -bit hash function h takes an arbitrary message m and produces a string of n bits. As with every good hash function, we want h to distribute messages among the 2^n possible digests. It should also be fast to calculate.

To be a *cryptographic hash function*, h must satisfy these properties:

- 1) **Preimage resistance:** Given a hash value v , it should be hard to find a message m such that $h(m) = v$.
- 2) **Second preimage resistance:** Given a message m_1 , it should be hard to find a message $m_2 \neq m_1$ such that $h(m_1) = h(m_2)$.
- 3) **Collision resistance:** It should be hard to find any pair of messages m_1, m_2 such that $h(m_1) = h(m_2)$.

WHAT IS “DIFFICULT” IN THIS CONTEXT?

The idea of collision resistance is a bit weird – we know by the Pigeonhole Principle that there will be many messages that all hash to the same value.

What we want is for it to be infeasible to *intentionally* generate collisions. If h produces no discernable patterns in its output, then the only approach left to any hackers is guess-and-check, and brute force will require $O(2^n)$ guesses to match a particular hash value.

But we need the output to be deterministic based upon the message, so there must be some pattern, which can then be exploited by an adversary to reduce the number of guesses needed. If the expected number of guesses is $O(2^s)$, then we say that the hash function has a *security strength* of s bits.

BIRTHDAY ATTACKS

Recall the Birthday “Paradox”: How many people do we need in a room before the odds are two of them share a birthday?

It’s a much smaller number than intuition suggests because we didn’t require matching a specific birthday. In much the same way, general collision resistance is harder than preimage resistance – the rule is that if you need g guesses to match a specific hash value, you expect to see some pair of guesses that have matching hash values after $\sqrt{g}$ guesses.

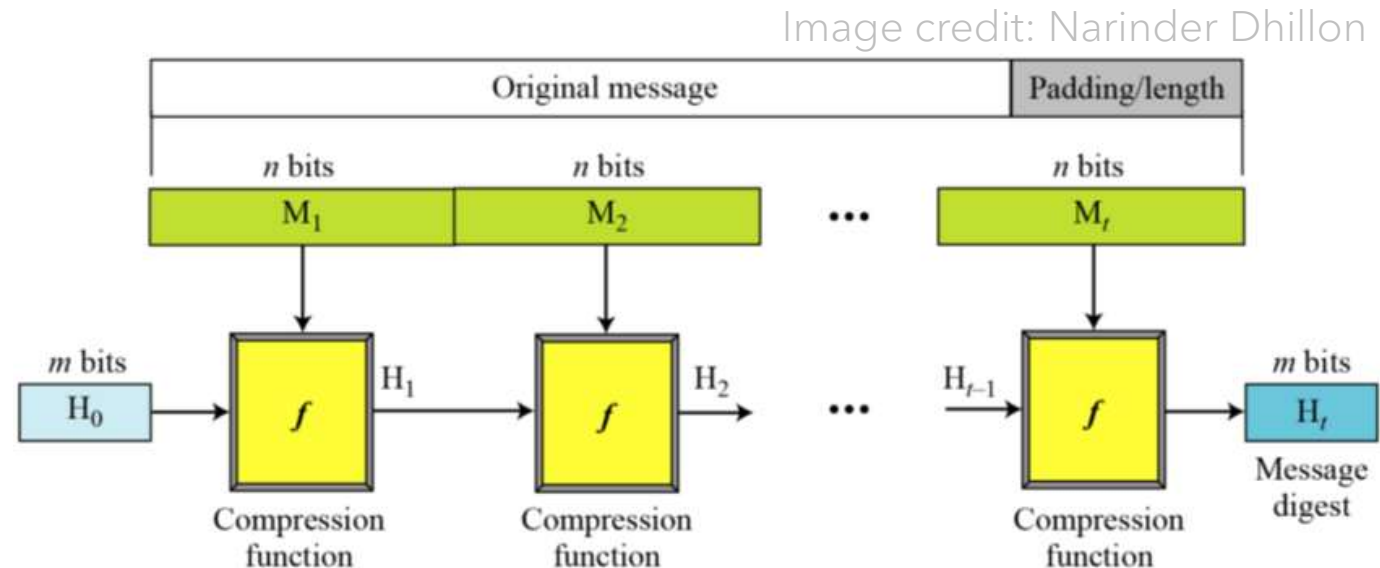
Thus, a function with s bits of preimage resistance will only ever have $s/2$ bits of collision resistance.

PRODUCING CRYPTOGRAPHIC HASH FUNCTIONS

To produce the desired properties, hash functions should have the *avalanche effect*: small changes in the message should produce large unpredictable changes in the output.

Ideally, changing any single bit of the input should have a 50% chance of changing each bit of the output.

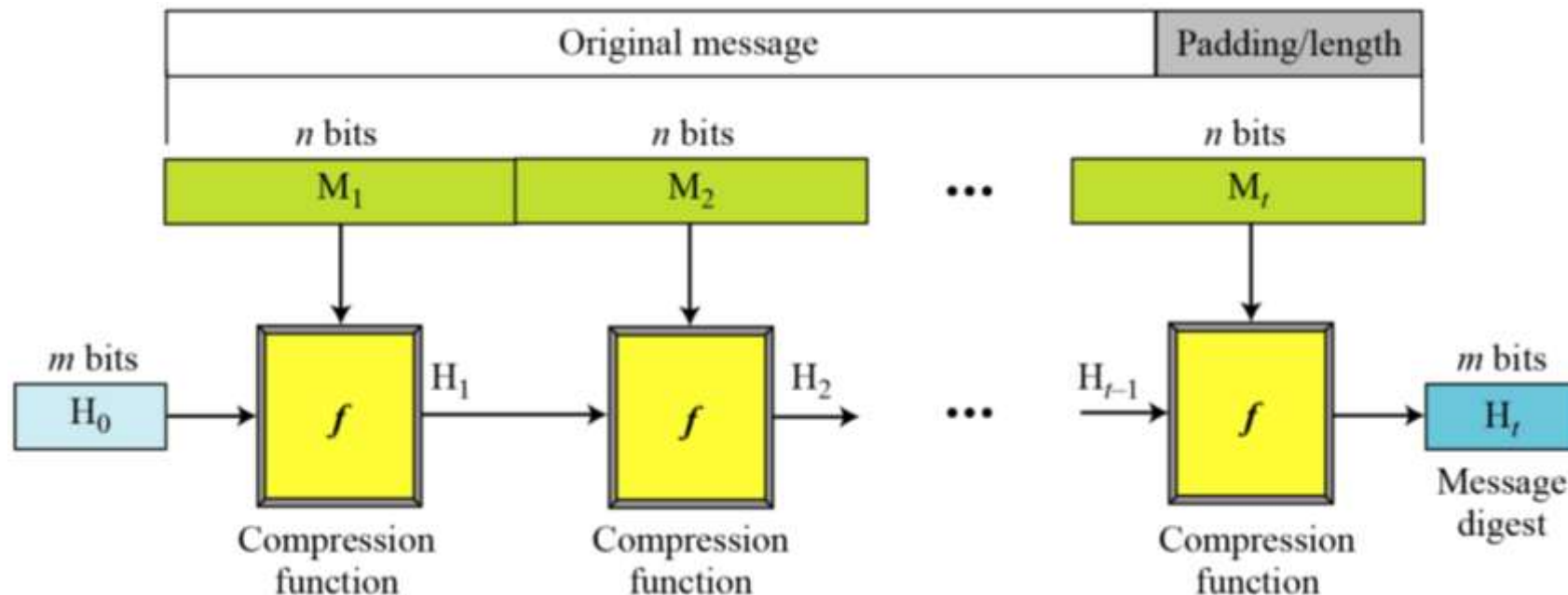
The technique most CHFs use to accomplish this is called the *Merkle-Damgård construction*. It breaks the message into blocks and feeds them in one by one.



WHAT'S IN THE BOX?

Most of the work here is done in those yellow boxes labeled “compression function”. These functions usually involve bitwise operations (often XOR and rotation), plus a set schedule of *round keys* that are combined with the message block.

Image credit: Narinder Dhillon



SPECIFIC EXAMPLE: MD5

MD5 is a **deprecated** hash function – but it serves as an effective example.

It uses 64 rounds of the operation depicted at right. The round keys K_i vary each round, as do the rotations

The function F changes every 16 rounds, using these functions:

$$F(B, C, D) = (B \wedge C) \vee (\neg B \wedge D)$$

$$G(B, C, D) = (B \wedge D) \vee (C \wedge \neg D)$$

$$H(B, C, D) = B \oplus C \oplus D$$

$$I(B, C, D) = C \oplus (B \vee \neg D)$$

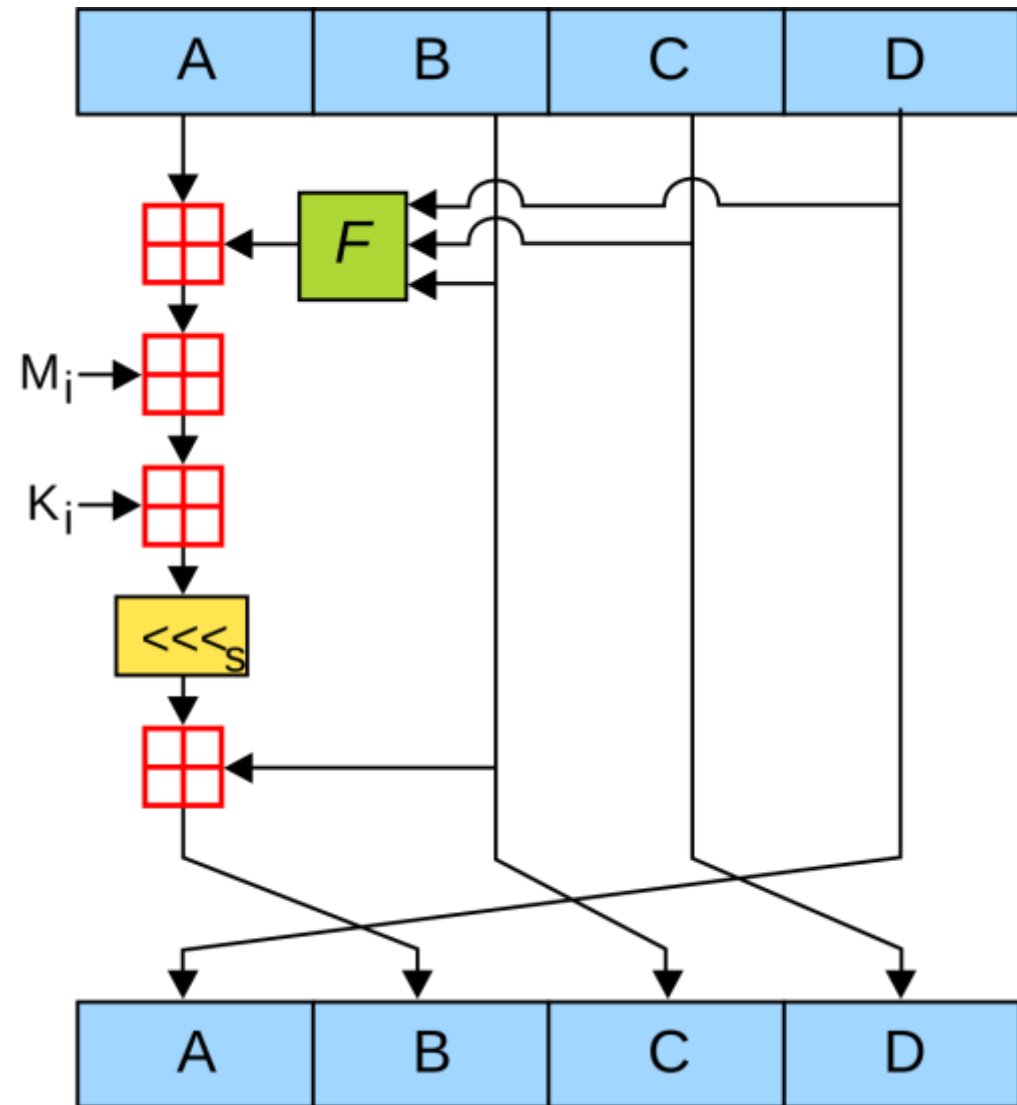


Image credit: Wikipedia user surachit

EXAMPLE APPLICATION: `/etc/passwd`

One place where hash functions are hugely important are in password files.

For a variety of reasons, systems should never **NEVER** store plaintext passwords anywhere.

But, we do need some way of verifying that the password entered matches the password the user set.

The solution? We stored *hashed* copies of each password.

If someone gets the hashed password file, they can't use that to log in.
(But having the hashed copies stolen is still a major security issue.)

DIGITAL SIGNATURES REVISITED

The described process doesn't do much for two key properties of signatures – confirming the identity of the signer and *nonrepudiation*: disallowing the signer to change their mind after signing.

These require a bit of *public-key cryptography* – a system of encoding messages wherein each entity has a pair of encryption/decryption keys, one kept secret and one publicized.

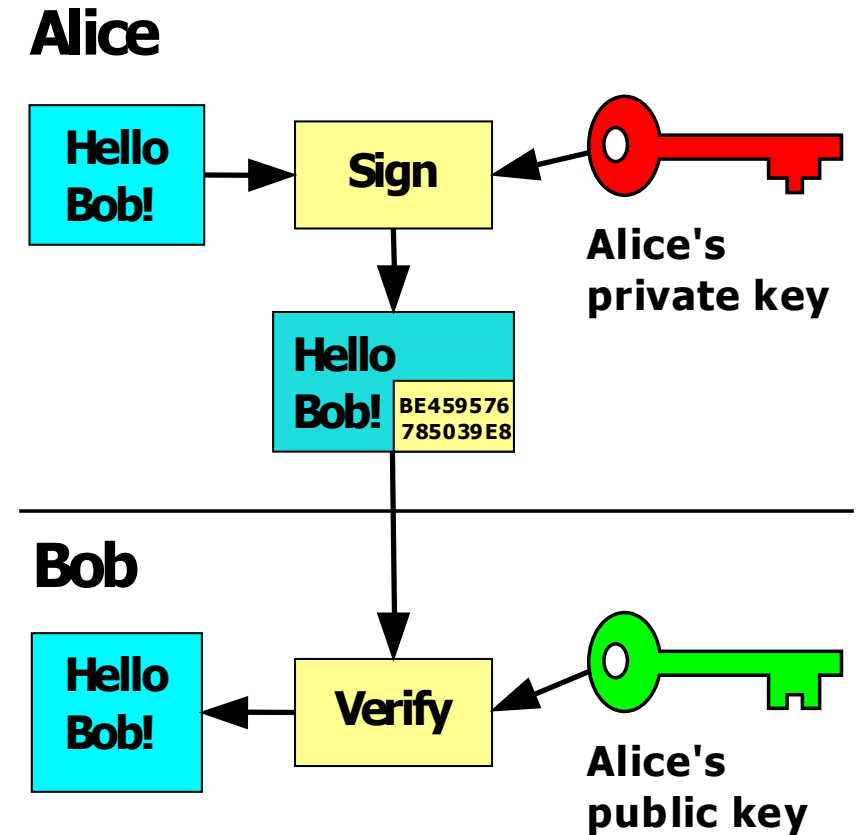


Image credit: Wikipedia user BrianjReinwald

DIGITAL SIGNATURES REVISITED

The easy way to generate a "signature" is to create a string like "My name is Alice and I approve this message." and then encrypt it with your private key.

Anyone who cares to can use your public key to decrypt the message.

The problem, of course, is that this signature can then be lifted and attached to OTHER messages.

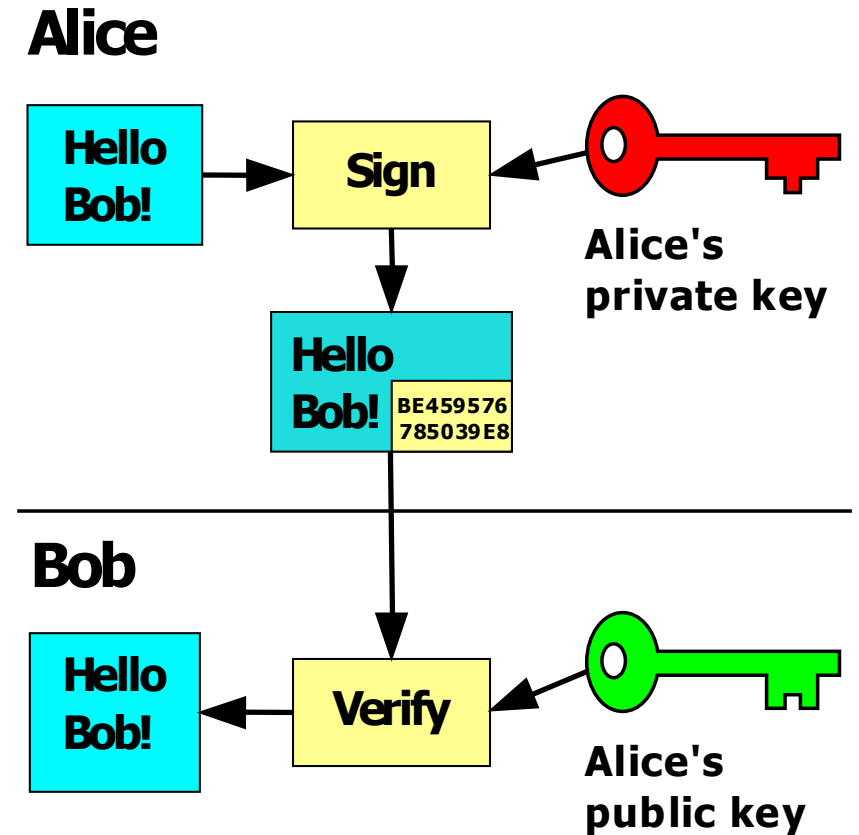


Image credit: Wikipedia user BrianjReinwald

DIGITAL SIGNATURES REVISITED

The solution, then, is to hash the message. The signature then becomes "My name is Alice, and I approve the message with hash key 0xA845B1E00..."

The signature is thus valid only for that message, unless one can find a different message with the same hash key. But a good CHF makes that infeasible.

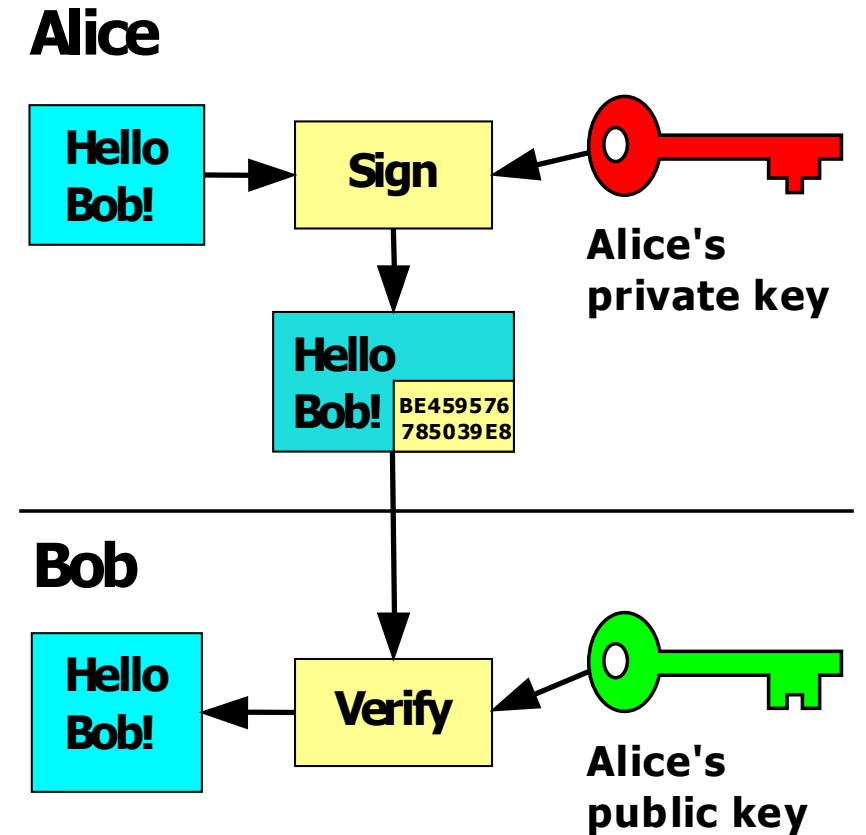
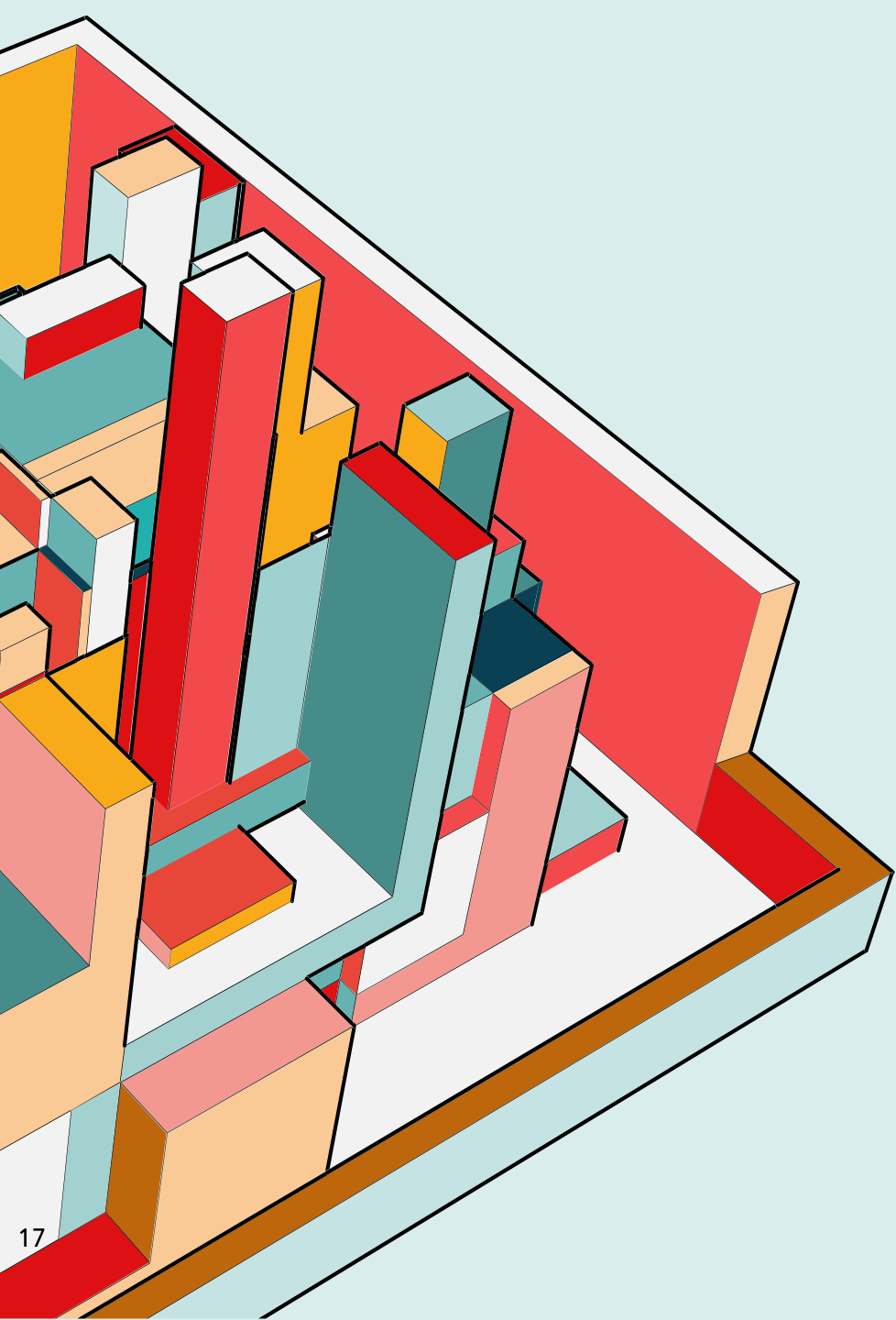


Image credit: Wikipedia user BrianjReinwald



EXAM REVIEW

LECTURE 9: GREEDY ALGORITHMS I

- General notion of a greedy algorithm
 - Proof of optimality – usually “get ahead & stay ahead” or an exchange argument
- Minimum spanning trees
 - Definition of tree & spanning tree
 - Prim’s & Kruskal’s algorithms (execution, runtime analysis)

LECTURE 10: GREEDY ALGORITHMS II

- Shortest path & Dijkstra's algorithm
 - Relation to breadth-first search
 - Execution using a priority queue
 - Restrictions on the input graph
 - Runtime analysis
- Huffman codes
 - Key notion: Prefix-free encoding
 - Bottom-up construction of Huffman tree

LECTURE 11: SHORTEST PATH CONT'D; HEAPS

- Bellman-Ford algorithm
 - Problems it solves / problems it doesn't
 - Execution & runtime analysis
- Heaps
 - General concept / heap properties
 - Insert & pop operations
 - Initial building of heap (top-down, bottom-up)
 - Use of heaps to create priority queue
 - Array implementation of heaps

LECTURE 12: GREEDY APPROXIMATIONS

- NP-completeness (basic idea)
 - Reducing (i.e. transforming) one problem to another
- Greedy approximate solutions to hard problems
 - General idea: Prove that greedy solution is no worse than ____ times the optimal solution.
 - Sometimes this requires restricting the input, so that we can only work with some problem instances.
 - Examples: Vertex cover, traveling salesman

LECTURE 13: HASHING I

- Counting Sort
 - Execution & runtime analysis
- Hashing
 - The problem we're trying to solve: Support fast dictionary operations (insert, search, delete)
 - Trade space for time by using a modified direct-address scheme, with many keys mapped to each hash bucket.
 - Resolve collisions via chaining ("open hashing")
 - Good hash functions & average-case analysis
 - Universal hashing to approximate simple uniform hashing

LECTURE 14: HASHING II

- Closed hashing (“probing”)
 - Linear, quadratic, double-hashing
 - Execution and runtime analysis
- Other hashing applications
 - Bloom filters
 - Execution; likelihood of false positives
 - Locality sensitive hashing – analysis

COMPLETE THE POLL!

Left side is second Submittity poll: What are you most concerned about for Exam 2? Right side is standard end-of-class feedback form.

